SRW3 Phase 1K — Implementation Package Report

The verified SRW3 surface, assembled, certified, and runnable

Branch `phase1k-implementation-package` · additive tree `phase1k/` · frozen base `phase1j-security-closure` @ `a91dbcc` (`phase1j-complete`)

---

1. Objective and outcome

Phase 1K packages the verified SRW3 deliverable surface — the executable Python mirror driving the real frozen authorization gate, the golden conformance vectors, the adversarial and unit suites, the K krun transcripts consumed by the runners' cross-checks, and the K proof-tier sources — into one self-contained, byte-identity-certified, independently runnable implementation package at `phase1k/package/`.

Outcome (Level III): DELIVERED — the three-artifact completion gate is met from the package tree itself, including a fresh K/LLVM tier. The package tier executed fresh: 1J unittests 27/27, the 1J adversarial matrix 48/48 (including the reachability enumeration 3474 committed prefixes / 386 unsafe derived states caught / 0 committed unsafe), R1 48/48 (with the K↔Python KX cross-check against the packaged K transcripts), R2 87/87, R3 181/181 (with KX), 1E/1F/1G suites all green — and the byte-stability oracle (`verify_all.sh`) proved that the golden vectors REGENERATED during the run are byte-identical to the packaged, K-certificate-conformant copies (55/55 manifest entries stable). On top, the FULL 1J LLVM suite (real Ethereum Keccak-256) ran fresh from the interpreter built out of the PACKAGE's own K sources, reproducing the frozen T0/T2/T6 transcripts byte-for-byte (T0 certificate, honest-pipeline COMMITS, unsafe-composition REJECTS, self-computed anchor). Independently, the frozen trees themselves re-ran green in-repo with `git diff a91dbcc HEAD` outside `phase1k/` EMPTY.

The phase definition is ASSUMED and this is stated first: no written Phase 1K specification exists anywhere in the repository (verified by `git grep` over all reachable history for `phase ?1k`, `1K implementation`, `implementation package`; the repo worklog ends at the 1I-R3 section). The interpretation follows the established phase pattern: an isolated additive branch carrying the deliverable with proof reports, concrete executions,

and regression results available for independent audit — no semantic change, no new proof obligation, frozen history untouched. If the intended 1K differs, the package is additive and revertible; no frozen byte is affected.

2. What was built

`phase1k/package/` (55 files):

- **The Python runtime surface, byte-identical** — `python-gen` (the 1D-R1 lineage verifier core), `phase1e`..`phase1j` python modules (authenticated state, execution evidence, THE FROZEN GATE `authz_model.verify_lineage_g`, R1/R2/R3 machines and runners, the J machine mirror), the 1J unittest suite, and the vectors (`j_golden.json`, the R3 configuration-root vectors). Every copied file is sha256-certified identical to its frozen source at the base commit (`executions/byte-identity.txt`: 48/48).
- **The K proof-tier sources, byte-identical** — under `semantics/`: the frozen 1G gate definition and its five kprove modules, the R3 machine with its 50-claim ladder and runner, and the J machine with its 42-claim ladder and both runners. The three frozen K krun transcripts that the Python runners parse for the KX cross-checks (R1 T2/T5, R3 byte certificate) are packaged byte-identically — they are part of the cross-layer surface, not decorative.
- **Seven additive artifacts** — `VERSION`, `README.md` (integrator entry point), `INTEGRATION-GUIDE.md` (surface reference, input model, fail-closed semantics, non-claims), `CLAIMS-INDEX.md` (the reconciled proof inventory 1E..1J with statuses transcribed verbatim from the frozen records), `install.sh`, `run_all.sh`, `verify_all.sh`, plus `MANIFEST.sha256` (55 entries) as the package's integrity oracle.

Three entry points, each doing exactly what its header says:

```
./install.sh # interpreter probe (incl. coincurve),
# path-compat layer, self-integrity
./run_all.sh [OUT-DIR] # the full Python verification tier
./verify_all.sh # byte-stability oracle (incl. regenerated vectors)
```

3. The fresh executions (completion gate, leg 2)

All tiers were executed FROM THE PACKAGE TREE (`run_all.sh`), transcripts in `phase1k/executions/`:

tier	transcript	result
1J unittests	`unittests_1j.txt`	27 tests OK
1J adversarial matrix + vectors	`attacks_1j.txt`	**48/48 PASS**; `j_golden.json` regenerated in place

tier	transcript	result
R1 binding + KX	`regression_r1.txt`	**48/48 PASS** (exit 0)
R2 provenance	`regression_r2.txt`	**87/87 PASS**
R3 config-root + KX	`regression_r3.txt`	**181/181 PASS**
1E authenticated state	`suite_1e.txt`	OK
1F execution evidence	`suite_1f.txt`	16/16 + 3/3 byte
1G frozen gate	`suite_1g.txt`	25/25
byte-stability oracle	`verify_all.txt`	55/55 files stable incl. the regenerated vectors

The KX groups deserve emphasis: R1's runner parses the K-side kore preimage terms out of the packaged T2/T5 krun transcripts and requires byte equality with the Python-computed preimages; R3's runner does the same against the packaged LLVM byte-certificate transcript. Both passed from the package — the K↔Python cross-layer conformance travels with the package.

The golden-vector regeneration is the package's strongest runtime statement: `run_all.sh` rewrites `phase1j/vectors/j_golden.json` from the packaged mirror alone, and `verify_all.sh` then requires the manifest to still match. A pass simultaneously proves mirror determinism across runs and byte-identity with the values the K/LLVM certificate produced at 1J.

4. Regression results (completion gate, leg 3)

Two independent regression tiers, both green:

- **Package tier** (the table above): the frozen suites re-run from the package against their exact expected counts.
- **Frozen-tree tier** (`executions/frozen-regression.txt` + `frozen_*.txt`): the same suites re-run from the repo's own frozen trees at `a91dbcc` — identical results (48/48, 87/87, 181/181, 16/16+3/3, 25/25, 27 OK), with `git diff a91dbcc HEAD` outside `phase1k/` EMPTY and the frozen worktree CLEAN after the runs. The 1J vector regeneration inside the frozen tree reproduced byte-identically (git saw no change), independently confirming determinism.

5. The K/LLVM tier — fresh concrete execution with real keccak

The pinned toolchain (K v7.1.337, z3 4.13.3, LLVM-15 chain 15.0.6-4+b1, libsecp256k1, KEVM v1.0.921 sources, blockchain-k-plugin @ 207ae512) was rebuilt in this environment from the repo's recorded recipe after the sandbox reset. On top of it, this phase performed the **full fresh K/LLVM tier against the PACKAGE's own K sources** (`executions/k_tier/`):

- The J demo definition was kompiled from `package/semantics/phase1j/semantics/srw3closure-j-demo.k` (the packaged bytes — certified identical to the frozen tree) with the multiblock-fix krypto shim: `kompile --backend llvm --hook-namespaces KRYPTO` exit 0; interpreter linked (`build_llvm.log`).
- The FULL 1J demonstration + attack suite ran fresh with REAL Keccak-256: 14 executed demos (T0–T13) + 5 kparse ill-formedness rejections, runner exit 0 (`run_demos_stdout.txt`, `llvm/`).
- **Byte-identity of the fresh concrete executions with the frozen 1J transcripts**: T0 (the cross-layer certificate — catalog canon, all roots, evidence identity, block commitment, computed verdicts, SafeJ), T2 (the honest two-block pipeline COMMITS), and T6 (the unsafe 3-app composition REJECTS, state/head/counter unchanged) are all IDENTICAL to the frozen `phase1j/transcripts/llvm/` records modulo header lines, and the self-computed authenticated anchor (`llvm/anchor.txt`) is identical — the machine re-derives the same anchor from the same configuration.
- Toolchain deviations of record (this rebuild): `libclang-common-15-dev` extraction is required (clang resource headers) and `flex` must be on PATH; the R2 recipe's `llvm-kompile-clang` patch is wrong for K v7.1.337 and was replaced by the pristine driver with one surgical path substitution — all recorded in `executions/k_tier/README.md`.

The kprove ladders (hs backend) were NOT re-run fresh; their authoritative logs of record live in the frozen trees and are reconciled in `package/CLAIMS-INDEX.md`. This is disclosed; the package's K-tier claim rests on the fresh LLVM executions plus the frozen logs.

6. The compat layer and other disclosed mechanics

Three environment realities are handled WITHOUT editing a frozen byte:

1. **Hardcoded historical paths.** The frozen 1E/1F/1G test files and the R2 runner hardcode `/home/z/my-project/srw3-kevm/...` and `/home/z/my-project/srw3b-work/...`. `install.sh` symlinks exactly those two names to the package root (refusing to clobber by default; `--adopt-quarantine` moves pre-existing trees aside). On a fresh host the links are created; where the names exist and the operator declines, `--no-compat` keeps the relocatable tiers runnable and the failure of the hardcoded tiers is disclosed — not hidden.
2. **Stale shadowing trees.** This container's pre-reset leftovers (`srw3-kevm/` 0-D/1C/1D era, `srw3b-work/` 1B era) would win the import

race against the package through the hardcoded inserts (stale bytes silently shadowing verified ones). They were quarantined by `install.sh --adopt-quarantine` — an environment repair recorded in the audit §F.

3. **PYTHONPATH and CWD.** The frozen suites find siblings via PYTHONPATH (as the historical driver did), and the R1 runner resolves its KX transcripts relative to the CWD (so `run_all.sh` invokes it from its own directory). Both mechanics are visible in `run_all.sh` itself.

7. Assumptions, exclusions, non-claims

The package inherits every frozen assumption and non-claim verbatim (1J audit §E; `INTEGRATION-GUIDE.md` §6): the authenticated protocol anchor is external (governance out of scope); keccak-256 collision resistance is assumed; closure holds for the declared finite effect/policy fragment (Option A; Option B open); the universal reachability induction is NOT mechanized (concrete substitutes delivered); arbitrary-policy closure is NOT mechanized; no consensus integration; Level III only.

Specific to 1K: the phase definition is ASSUMED (§1); the package adds no theorem and modifies no semantics; the compat layer is an environment fixture, not an endorsement of hardcoded paths; the package does not re-vendor the K toolchain.

8. Artifact index

```
phase1k/
package/ THE implementation package (55 files)
VERSION README.md INTEGRATION-GUIDE.md CLAIMS-INDEX.md
install.sh run_all.sh verify_all.sh MANIFEST.sha256
python-gen/ phase1e..phase1j python surfaces (byte-identical)
phase1i-r1/transcripts/ phase1i-r3/transcripts/crosslayer/
phase1j/tests/ phase1j/vectors/ phase1i-r3/vectors/
semantics/{phase1g,phase1i-r3,phase1j}/{semantics,proofs}/
executions/ fresh transcripts (the completion-gate evidence)
byte-identity.txt 48/48 copied artifacts IDENTICAL + 7 ADDITIVE
unittests_1j.txt attacks_1j.txt regression_r{1,2,3}.txt suite_{1e,1f,1g}.txt
verify_all.txt the byte-stability oracle transcript
frozen-regression.txt + frozen_*.txt the in-repo frozen-tree tier
k_tier/ the fresh K/LLVM tier (build log, full T-series,
README with the toolchain deviations of record)
audit/IMPLEMENTATION-PACKAGE-AUDIT.md sections A-G
report/SRW3-Phase1K-Implementation-Package-Report.{md,pdf}
provenance/{MANIFEST.txt,provenance-hashes.txt}
```